Domain Adaptation

for Cost Model Training using DANN

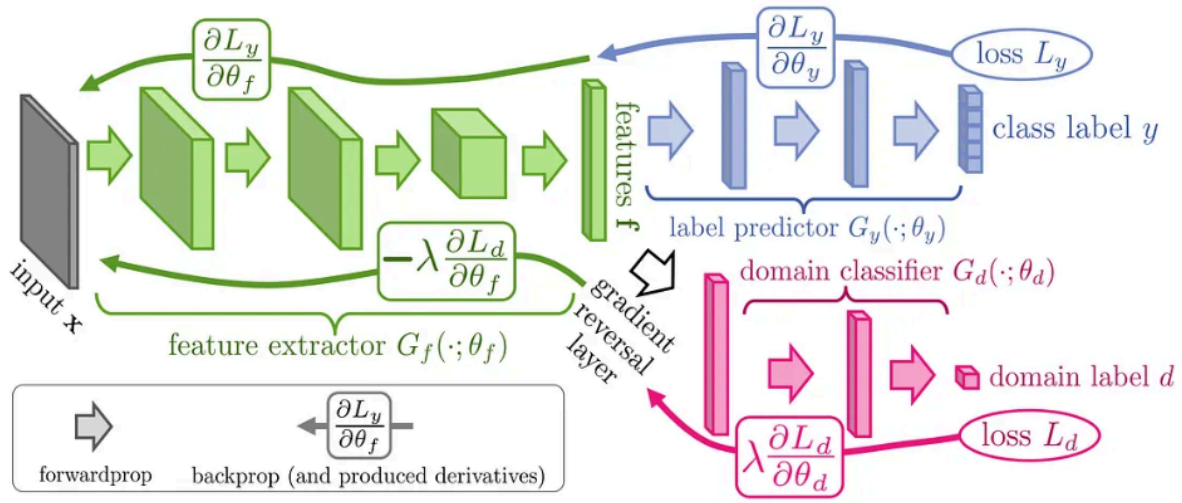
Motivation: Collecting labeled data for cost model training is time-consuming.

1. Domain-Adversarial Neural Networks (DANN)

DANN is a transfer learning framework that enables a model trained on a labeled source domain to generalize to an unlabeled (or sparsely labeled) target domain. It does so by simultaneously optimizing task performance while making learned features domain-invariant.

1.1 Architecture Components

DANN consists of three modules that are trained jointly:



Module	Symbol	Role

Feature Extractor	*Gf (θf)*	Maps raw input x into a shared latent representation z
Label Predictor	*Gy (θy)*	Regressor: predicts latency from z (trained on source only)
Domain Classifier	*Gd (θd)*	Classifies whether z came from source or target domain

1.2 Training Objective

DANN optimizes a minimax objective that balances task accuracy and domain confusion:

$$E(\theta f, \theta y, \theta d) = (1/n) \Sigma Ly(\theta f, \theta y) - \lambda \cdot [(1/n) \Sigma Ld(\theta f, \theta d) + (1/n') \Sigma Ld(\theta f, \theta d)]$$

Where:

- Ly — Task-specific loss (e.g., MSE for regression)
- Ld — Domain classification loss (binary cross-entropy: source vs. target)
- λ — Hyperparameter controlling the trade-off between task performance and domain invariance

The Gradient Reversal Layer (GRL) sits between Gf and Gd. During backpropagation it negates the gradient by −λ, forcing Gf to produce features that confuse the domain classifier while still minimizing task loss.

We adapt this to Cost Model Training because it is the key bottleneck in our project which requires 12000 data points in the current approach.

2. Two Domain Adaptation Scenarios - Cost Model

Depending on the deployment context, two distinct adaptation scenarios arise. Each differs in which features change across domains and requires a tailored strategy.

Scenario A	**Scenario B**
Different DB Engine, Same Servers	Same DB Engine, Different Servers
e.g., Postgres → MySQL on the same hardware	*e.g., Postgres on Server 1 → Postgres on Server 2*

2.1 Scenario A — Different DB Engine, Same Servers

Feature	Value

Internal Metrics	Different (engine-specific counters differ between Postgres / MySQL)
Workload Features	Same (query patterns are engine-agnostic)
Query Plans	Different (each optimizer produces its own plan format)
Knob Names	Different (e.g., shared_buffers vs. innodb_buffer_pool_size)

Key Challenge: Implicit Domain Leakage

If raw knob names are passed directly to Gf, the feature extractor may implicitly encode domain information — essentially letting the domain classifier 'cheat' by detecting engine-specific parameter names. This breaks the adversarial objective.

Example: 'Feature X only appears in Postgres' → Gd trivially succeeds → Gf cannot make features domain-invariant → DANN fails.

Solution: Semantic Feature Normalization

Map engine-specific configuration knobs to common semantic features before feeding them to Gf:

- Replace knob names with semantic equivalents (e.g., memory_buffer_size instead of shared_buffers or innodb_buffer_pool_size)

Model Setup

Component	Configuration
Regressor (Gy)	Predicts query latency; Hardware configs balanced 50% S1 / 50% S2
Domain Classifier (Gd)	Binary: Postgres (0) vs. MySQL (1)
Primary Metric	Domain Classifier Accuracy

2.2 Scenario B — Same DB Engine, Different Servers

Feature	Value
Internal Metrics	Same (same engine produces the same counter types)
Workload Features	Same
Query Plans	Same
Knob Names	Different (hardware differences may alter tunable params)

Model Setup

Component	Configuration
Regressor (Gy)	Predicts latency; Hardware configs balanced 50% S1 / 50% S2
Domain Classifier (Gd)	Binary: Server 1 (0) vs. Server 2 (1)
Regression Metrics	MSE / MAE / RMSE

3. Unsupervised Domain Adaptation (UDA) — Analysis

3.1 Standard DANN Data Flow

Source (Postgres): (x_s, y_s) ← fully labeled
Target (MySQL): (x_t) ← unlabeled

Training objective:

- Task loss: MSE(Gy(Gf(x_s)), y_s)
- Domain loss: CrossEntropy(Gd(Gf(x)), domain_label) applied to both domains

Expected outcome: Gf learns domain-invariant features; Gy learns latency mapping from source; the hope is that this mapping transfers to the target.

3.2 Core Limitation — Concept Shift

UDA assumption: P(Y | X) is similar across domains
Reality for DB engines: Same workload + config → different latency in Postgres vs. MySQL
Therefore: P_S(Y | X) ≠ P_T(Y | X) → Concept Shift

Even if DANN perfectly aligns feature representations (z_pg ≈ z_mysql), the predictor Gy still learned to approximate Postgres latency. It may therefore systematically mispredict MySQL latency.

3.3 When UDA Works Well

- **Hardware changes (same DB engine, different servers)** ✓
- **Workload distribution shifts** ✓
- **Config range differences** ✓

3.4 When UDA is NOT Sufficient

- **Cross-DB adaptation (Postgres → MySQL) — execution engine, optimizer, and storage all differ ✗**
- **You fixed this by:**
  - **Gy(z, db_type)**
  - **Now model learns:**
  - **latency = f(z, db_type)**

4. Semi-Supervised Domain Adaptation (SSDA)

SSDA addresses the concept shift limitation by incorporating a small number of labeled target samples alongside the full labeled source and unlabeled target data.

4.1 Data Setup

Source (Postgres): (x_s, y_s) ← full labels
Target (MySQL) — unlabeled: (x_t) ← majority of target data
Target (MySQL) — labeled: (x_{t_l}, y_{t_l}) ← small labeled subset

4.2 Training Objective

Three losses are combined:

$$L = L_{source} + \alpha \cdot L_{target} + \beta \cdot L_{domain}$$

L_source: MSE on Postgres labeled data
L_target: MSE on the small MySQL labeled subset
L_domain: Domain classifier loss (DANN adversarial)

4.3 Why SSDA Works — Intuition

Without target labels	With small target labels
Model incorrectly assumes P(Y|X) is the same across DBs — concept shift is unaddressed.	The model learns how MySQL differs from Postgres — correcting systematic prediction bias.

4.4 Final Architecture

z = Gf(x) → shared representation
y = Gy(z, db_type) → latency prediction (with DB type conditioning)
d = Gd(GRL(z)) → domain classification via Gradient Reversal

Training data usage per loss:

- Source data → task loss + domain loss
- Target unlabeled → domain loss only
- Target labeled → task loss + domain loss

4.5 Additional Enhancements

Residual Learning (Powerful)

Instead of learning MySQL latency from scratch, learn only the delta from Postgres:

latency_mysql = latency_pg + Δ_mysql

Where Δ_mysql is learned from the small labeled target dataset. This exploits strong source supervision while adapting to the target distribution efficiently.

5. Summary

Scenario	Approach	Works For	Key Limitation
Diff Engine, Same Servers	SSDA + semantic normalization	Cross-engine transfer with few labels	Concept shift (P(Y|X) differs)
Same Engine, Diff Servers	UDA or SSDA	Hardware / workload / config shift	Minimal — P(Y|X) is more stable